

JIT и AOT компиляция: ReadyToRun и NativeAOT

C# Developer. Advanced



Проверить, идет ли запись

Меня хорошо видно & слышно?





Елена Сычева

Team Lead Full-Stack Developer

- Образование: СПбГУ (Прикладная математика), СПбГУ(псих. фак. программа "Преподаватель высшей школы")
- Более 20 лет опыта работы разработчиком (C#, Angular, .Net, React, NodeJs)
- Преподаватель курса "C# Basic", "C# Professional", "Asp.Net", "Angular" в ОТУС

 @lentsych

 lentsglobal@gmail.com



Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в учебной группе **#канал**
группы



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Маршрут вебинара

Постановка целей

Основы исполнения .NET-кода

ReadyToRun

NativeAOT

Практика

Основные тезисы

Цели вебинара

К концу занятия вы сможете

1. Понимать разницу между JIT (Just-In-Time) и AOT (Ahead-Of-Time) компиляцией

2. Использовать ReadyToRun для ускорения холодного старта приложений

3. Применять NativeAOT для создания нативных самодостаточных исполняемых файлов с минимальным размером и мгновенным запуском.



Введение

Процесс компиляции

- С# компилятор (Roslyn) преобразует наш код в IL (Intermediate Language) — промежуточный язык
- Создается сборка (.dll/.exe) содержащая IL-код + метаданные
- При запуске CLR (Common Language Runtime) загружает эту сборку

Проблема "холодного старта"

- При первом вызове TransformData:
 - CLR обнаруживает, что метод еще не скомпилирован
 - JIT-компилятор активируется
 - IL преобразуется в нативный машинный код
 - Машинный код кешируется в памяти
- При последующих вызовах:
 - Используется уже готовый машинный код из кэша
 - JIT-компилятор не участвует

```
public class ColdStartDemo
{
    public void ProcessData()
    {
        // ПЕРВЫЙ ВЫЗОВ - медленный
        var data1 = TransformData("hello"); // ⌚ Задержка!

        // ПОСЛЕДУЮЩИЕ ВЫЗОВЫ - быстрые
        var data2 = TransformData("world"); // 🚀 Быстро!
        var data3 = TransformData("!");      // 🚀 Быстро!
    }
}
```

AOT

- AOT – компиляция (части) кода программы сразу в нативный код, до запуска приложения



AOT tools overview

Дополняющие JIT:

- Ngen (legacy, .NET Framework)
- **ReadyToRun (crossgen2)**
- Mono AOT

Нативные:

- Mono Full-AOT
- IL2CPP
- Burst (Unity)
- **Native AOT (ex. CoreRT)**
- LLVM backend
- UWP (.NET Native)



Основы исполнения .NET-кода

Common Language Runtime (CLR) и Common Language Infrastructure (CLI)

Common Language Runtime (CLR)

- JIT-компилятор,
- Сборщик мусора (GC),
- Управление типами,
- Обработка исключений,
- Безопасность типов

Common Language Infrastructure (CLI)

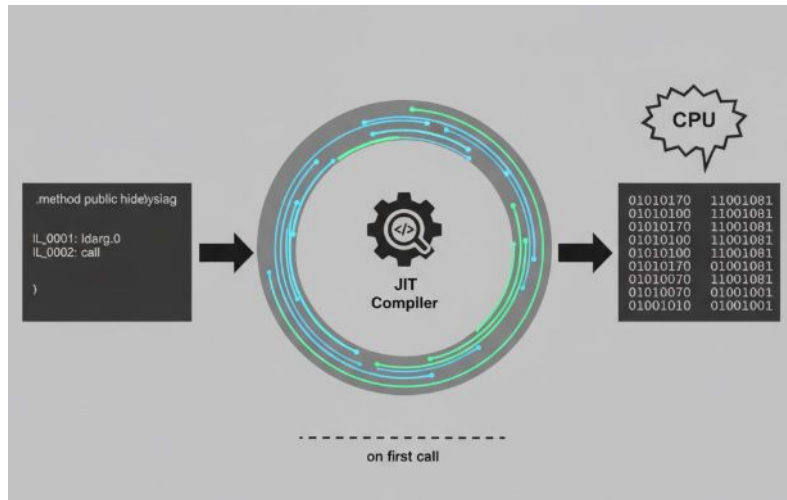
- Спецификация,
- IL,
- CLR,
- Base Class Library (BCL).

JIT-компиляция

JIT расшифровывается как 'Just-In-Time' – 'точно в срок' или 'прямо во время выполнения'.

При первом вызове метода JIT-компилятор «перехватывает» этот метод. Он берёт IL-код этого конкретного метода и компилирует его в нативный машинный код, который может быть выполнен центральным процессором (CPU) вашей машины. Этот скомпилированный машинный код затем кэшируется.

При последующих вызовах того же метода CLR просто использует уже скомпилированный машинный код из кэша.



JIT-компиляция

Оптимизации во время выполнения (Runtime Optimizations): CLR собирает данные о работе и адаптирует код.

Адаптивность к конкретному оборудованию: JIT генерирует машинный код под конкретный CPU.

'Холодный старт' (Startup Time):

Задержка при первом запуске из-за компиляции.

Накладные расходы: JIT-компилятор потребляет ресурсы (CPU, память) во время работы.

Виды JIT

- Foreground JIT – блокирующий, при первом обращении к
- типу/методу
- Main Thread Foreground – то же, но на основном потоке
- Multicore JIT Background – предиктивная компиляция на основе данных от предыдущих запусков (ProfileOptimization)
- Tiered Compilation Background – перекомпиляция часто используемых методов ради повышения эффективности кода (TieredCompilation)



Основные особенности JIT-компиляции в последних версиях .NET

- Многоуровневая Компиляция (Tiered Compilation) Уровни (Tiers): Компиляция кода разбита на два основных уровня:
 - Tier 0 (Быстрая JIT-компиляция): При первом вызове метода JIT компилирует его максимально быстро, без дорогостоящих оптимизаций (Quick JIT). Цель — как можно скорее получить исполняемый код, чтобы приложение начало работать. Код, сгенерированный на этом уровне, не самый быстрый, но он быстро готов.
 - Tier 1 (Оптимизированная JIT-компиляция): Если метод вызывается достаточно часто ("горячий" метод), среда выполнения (CLR) повторно компилирует его в фоновом режиме с использованием полного набора оптимизаций. Этот код значительно быстрее, но его генерация занимает больше времени. После завершения компиляции Tier 1 перекомпилированный код заменяет менее оптимизированный код Tier 0.

On-Stack Replacement (OSR)

Expert-деталь: почему многоуровневая компиляция не «ломает» длинные циклы

Ситуация

Метод вызван всего 1 раз, но внутри — цикл на миллионы итераций.



Без OSR

Код «застрял» бы в медленном Tier 0 до конца цикла — Tier 1 подключается только при следующем вызове.



С OSR (с .NET 7)

CLR подменяет код метода на оптимизированный Tier 1 ПРЯМО во время исполнения цикла.

Источник: [dotnet/runtime #64659](#). OSR включён по умолчанию с .NET 7; Dynamic PGO — по умолчанию с .NET 8.



Основные особенности JIT-компиляции в последних версиях .NET

- Оптимизации, Специфичные для Архитектуры (SIMD и ISA)
 - Современный JIT-компилятор активно использует специальные наборы инструкций (Instruction Set Architectures, ISA) конкретных процессоров для генерации более быстрого кода.
 - SIMD (Single Instruction, Multiple Data)
 - Автоматическая векторизация
 - Поддержка AVX-512 (в .NET 8+) и AVX10.2 (в .NET 10): введена поддержка новейших и наиболее широких расширений ISA, которые обеспечивают ещё более высокую производительность в числовых вычислениях.
 - Улучшения Arm64 (в .NET 8+): Продолжающиеся улучшения в генерации кода для архитектуры Arm64, что критично для облачных и мобильных приложений.

Основные особенности JIT-компиляции в последних версиях .NET

- Оптимизация Структур и Общих Операций
 - Физическое Продвижение (Physical Promotion): Улучшенная оптимизация, при которой члены небольших структур помещаются непосредственно в регистры процессора вместо стека. Это устраняет ненужные обращения к памяти, значительно ускоряя передачу структур в качестве аргументов или работу с ними как с локальными переменными.
 - Отмена Абстракции (Deabstraction): JIT может "отменять" абстракции, такие как `Array.AsReadOnly()` или `List<T>.AsReadOnly()`, удаляя избыточные проверки и обертки времени выполнения.
 - Улучшенный Инлайнинг (Inlining): JIT постоянно совершенствует логику, определяющую, когда следует встроить (inline) вызов небольшого метода непосредственно в тело вызывающего метода. Агрессивный, но умный инлайнинг является одним из главных источников повышения производительности.

Основные особенности JIT-компиляции в последних версиях .NET

- Динамическая PGO (Dynamic PGO): начиная с .NET 6, а по умолчанию — с .NET 8, JIT собирает профили реального выполнения (например, какие ветки if/else выполняются чаще) во время работы кода Tier 0.
- Эти данные затем используются для Tier 1 компиляции. JIT может, например, оптимизировать код, основываясь на наиболее вероятных путях выполнения, используя более эффективное расположение кода (layout) и устраняя неиспользуемый код (dead code elimination).
- Это позволяет JIT принимать оптимизационные решения, которые невозможно сделать с помощью статического анализа.

Основные особенности JIT-компиляции в последних версиях .NET

- Улучшения в Области Потребности и Памяти
 - Оптимизированный доступ к ThreadStatic полям: В .NET 8 и более поздних версиях улучшена генерация кода для доступа к полям, помеченным [ThreadStatic], что делает такие операции более быстрыми.
 - Повышенная Пропускная Способность (Throughput): Непрерывные улучшения в самом компиляторе, которые позволяют ему компилировать код быстрее, используя меньше ресурсов процессора.

Измерения

- При замерах результативности AOT важны только метрики приложения
- ETW и профилирование – только вспомогательные инструменты

Вопрос аудитории

**Короткоживущая CLI-утилита запускается тысячи раз в CI.
Что важнее для неё?**

A

Скорость холодного старта

B

Пиковая производительность

Пишите A или B в чат. Обсудим, как ответ влияет на выбор JIT / R2R / NativeAOT.



ReadyToRun (R2R)

ReadyToRun (R2R) – формат данных, содержащих нативный код
для .NET assemblies



Crossgen2 и ReadyToRun

- AOT для современного .NET – общего назначения, т.е. работает в паре с JIT
- ReadyToRun (R2R) – формат данных, содержащих нативный код для .NET assemblies
<https://github.com/dotnet/runtime/blob/main/docs/design/coreclr/botr/readytorun-format.md>
- crossgen/crossgen2 – утилиты для генерации R2R данных
<https://www.nuget.org/packages/Microsoft.NETCore.App.Crossgen2.win-x64>
- ReadyToRun-образ – сборка, содержащая нативный код в R2R формате

Как это работает?

1. Компилятор заранее (Ahead-of-Time) выполняет работу JIT-компилятора и сохраняет результат в исполняемый файл.
2. При запуске, CLR видит, что для метода уже есть готовый машинный код, и загружает его, минуя этап JIT-компиляции.
3. Для методов, не скомпилированных в R2R, используется стандартный JIT.

Таким образом, R2R – это своего рода 'ускоритель' для JIT, но он не заменяет CLR полностью. Ваш приложение по-прежнему работает под управлением CLR.



ReadyToRun

- Значительное ускорение холодного старта (до 30-40%).
- Полная совместимость со всеми фичами .NET (Reflection, динамическая загрузка и т.д.).
- Простота публикации: `dotnet publish -c Release -r win-x64 -p:PublishReadyToRun=true` (флаг `--self-contained` — по желанию).

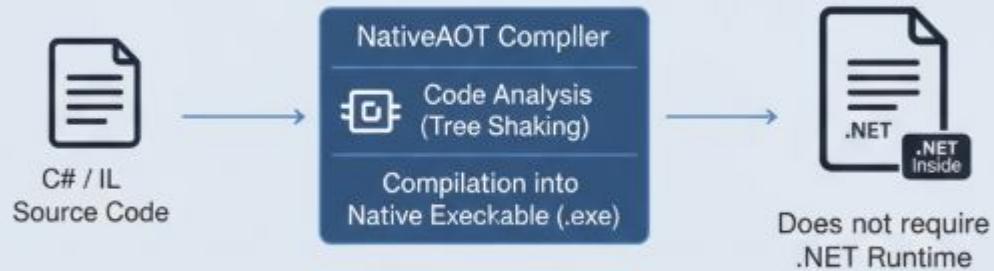
- Увеличение размера сборки
- Машинный код менее оптимизирован, чем "чистый" JIT в рантайме
- Кроссплатформенность ограничена: нужно публиковать отдельно для каждой RID (Runtime Identifier), например, win-x64, linux-x64.

ПРИМЕР

NativeAOT

AOT

Towards ExtriveAOT – Speed: - True Native Compilation



Goal: Maximum Speed & Minimum Size

NativeAOT берет весь ваш .NET-код, а также те части .NET Runtime (CLR) и библиотек (BCL), которые фактически используются вашим приложением, и компилирует их в один-единственный, автономный нативный исполняемый файл (.exe).



Что такое NativeAOT

- Это полная AOT-компиляция. Исходный код компилируется напрямую в единый нативный исполняемый файл.
- JIT-компилятор и сама CLR исключаются из финального артефакта.

Что такое NativeAOT

- NativeAOT (Ahead-Of-Time) компиляция — одно из самых быстро развивающихся направлений в .NET: зрелость в .NET 8/9 и production-ready для большинства API-проектов в .NET 10 (LTS, ноябрь 2025).
- Основная цель — не только сделать AOT-компиляцию возможной, но и обеспечить ее широкую применимость, улучшенную производительность и меньший размер исполняемого файла.

Как это работает?

- Специальный компилятор (на базе ILC — IL Compiler) анализирует всё приложение, определяет, какие типы и методы реально используются (Static Analysis), и генерирует из них нативный код.
- Результат — .exe (для Windows) или бинарный файл (для Linux), который запускается ОС напрямую

Усовершенствования NativeAOT в последних версиях .NET

- Расширение Поддерживаемых Сценариев и API
 - ASP.NET Core (Minimal APIs)
 - Entity Framework Core (EF Core) (Экспериментально): Добавлена экспериментальная поддержка для предварительно скомпилированных запросов LINQ.
 - GRPC и WCF: Улучшенная совместимость благодаря переходу на Source Generators вместо рефлексии для генерации необходимых RPC-клиентов и сервисов.
 - Улучшенная Совместимость с Общими Типами (Generics): включая улучшенную поддержку для ref struct в обобщенных параметрах в .NET 9.



Усовершенствования NativeAOT в последних версиях .NET

- Улучшения в Производительности и Размёре: новые версии направлены на улучшение пиковой производительности и уменьшение конечного двоичного файла.
- Агрессивная Обрезка (Trimming) и Уменьшение Размера: Компилятор NativeAOT стал более "умным" в определении неиспользуемого кода (dead code elimination). Он агрессивно удаляет не только неиспользуемые части фреймворка, но и метаданные, которые не нужны в нативном исполняемом файле.

Усовершенствования NativeAOT в последних версиях .NET

- Использование Новых Примитивов JIT/AOT: Улучшена генерация нативного кода, особенно для коллекций и типов-значений (struct), что часто превосходит результаты предыдущих JIT-компиляций.
- Новые Механизмы Безопасности: В .NET 9 реализована поддержка Runtime Enforced Code Integrity (RECI) и включена по умолчанию технология Control-flow Enforcement Technology (CET) для приложений в Windows.

Усовершенствования NativeAOT в последних версиях .NET

- Улучшенные Предупреждения AOT: Предупреждения, выдаваемые компилятором (например, IL3050 или IL2026), стали более точными и полезными.
- Поддержка UnsafeAccessor (в .NET 8+): Добавлен новый атрибут [UnsafeAccessor], который позволяет разработчикам получать доступ к непубличным членам (например, приватным полям) в сценариях AOT-компиляции.
- Улучшения в Отладке и Профилировании

Усовершенствования NativeAOT в последних версиях .NET

- Улучшенная Спецификация RID (Runtime Identifier): Требования к целевым системам стали более ясными. Например, двоичный файл AOT, созданный в Ubuntu 20.04, гарантированно работает в Ubuntu 20.04 и более поздних версиях, что упрощает управление зависимостями операционной системы.
- Расширенная Поддержка Кросс-Компиляции: Упрощены механизмы для кросс-компиляции (создания исполняемого файла для одной ОС/архитектуры на другой ОС/архитектуре) с использованием Docker-контейнеров и других инструментов.

Как это работает?

- Глубокий статический анализ и Tree Shaking.
- Полная AOT-компиляция. После того как компилятор определил весь необходимый код, он компилирует абсолютно все – ваш IL-код, нужные части CLR (такие как сборщик мусора, обработчик исключений), и используемые части Base Class Library – в единый нативный машинный код.
- Встраивание мини-рантайма. В итоговый .exe файл встраивается минимально необходимый набор сервисов CLR (сборщик мусора, поддержка потоков, обработка исключений и т.д.), но уже в нативном виде и только те части, которые нужны.

AOT

- Мгновенный запуск: Нет этапа JIT, нет загрузки CLR.
- Минимальное потребление памяти: В процессе нет JIT-компилятора, кэша JIT-кода и т.д.
- Минимальный размер: в сборку попадает только тот код, который реально нужен.
- Защита кода: IL отсутствует, что усложняет реверс-инжиниринг.

- Ограниченная поддержка Reflection
- Нет динамической загрузки сборок: `Assembly.LoadFrom(...)` не поддерживается.
- Нет поддержки некоторых платформенных фич
- Совместимость библиотек
- Требуется компиляции под каждую ОС/архитектуру

Настройка проекта для NativeAOT

```
<PropertyGroup>  
  <PublishAot>true</PublishAot>  
  
  <TrimMode>full</TrimMode>  
  
  <PublishTrimmed>true</PublishTrimmed>  
</PropertyGroup>
```

Публикация с NativeAOT

После настройки файла проекта и устранения проблем совместимости вы можете опубликовать ваше приложение.

Используйте команду `dotnet publish` с указанием целевой среды выполнения (RID). NativeAOT всегда требует явного указания RID, так как он генерирует нативный код для конкретной операционной системы и архитектуры.

```
dotnet publish -c Release -r <RID> /p:PublishAot=true
```

RID (Runtime Identifier)	Операционная система	Архитектура
win-x64	Windows	x64
linux-x64	Linux	x64
osx-x64	macOS	x64



Отличия NativeAOT и ReadyToRun

	ReadyToRun	NativeAOT
Задача	Ускорение старта	Нативное приложение
Размер артефакта	Больше	Меньше
Целевое приложение	Любое	Подготовленное
Рантайм	CoreCLR	Собственный

Как выбрать: JIT / ReadyToRun / NativeAOT

Нужны рефлексия, динамическая загрузка сборок, плагины?



JIT (+ R2R)

Нужен более быстрый старт без изменения кода?



ReadyToRun

CLI / контейнер, критичны старт + размер + память?



NativeAOT

Долгоживущий сервис → JIT/R2R. Мгновенный старт и малый размер → NativeAOT (с учётом ограничений).



Подготовка приложения к NativeAOT

1. Обратите внимание на предупреждения: Во время сборки вы можете увидеть предупреждения, например, IL2026 или IL3050. Они указывают на потенциальные проблемы, где Trim/AOT не может гарантировать безопасность.
2. Использование [DynamicallyAccessedMembers]: Если вы используете рефлексия, явно укажите, к каким членам (методам, полям и т. д.) будет осуществляться доступ, используя атрибуты [DynamicallyAccessedMembers] в .NET 7+.

```
[DynamicallyAccessedMembers(DynamicallyAccessedMemberTypes.PublicMethods)]  
public Type MyType { get; set; }
```


Подготовка приложения к NativeAOT

1. Использование Source Generators: Вместо ручного использования рефлексии, используйте Source Generators (например, для System.Text.Json) для генерации необходимого кода во время компиляции. Это предпочтительный подход для совместимости с AOT.

```
[JsonSourceGenerationOptions(WriteIndented = true)]  
[JsonSerializable(typeof(MyClass))]  
internal partial class AppJsonSerializerContext : JsonSerializerContext  
{  
}
```

Задание: «сломай и почини» NativeAOT

Шаги

1. Сериализуйте объект: `JsonSerializer.Serialize(obj)` — без контекста.
2. Соберите с `-p:PublishAot=true` → поймай предупреждения IL2026 / IL3050.
3. Почините через source generator: `[JsonSerializable(typeof(...))]` + `JsonSerializerContext`.
4. Убедитесь: предупреждения ушли, работает после AOT-публикации.

Что проверяем

Понимание, почему рефлексия несовместима с AOT и как source generators решают это на этапе компиляции.

Бонус: аннотируй оставшийся reflection-путь через `[DynamicallyAccessedMembers]`.

ПРИМЕР

Вопросы



если есть вопросы



если вопросов нет



Быстрый опрос

Serverless-функция — это всегда повод для NativeAOT?

Да

Нет, зависит

Подсказка: для Azure Functions команда рекомендует ReadyToRun, а не NativeAOT. Разбор — на следующем слайде.



Сценарии использования

- CLI-утилиты: Мгновенный запуск — ключевое требование.
- Serverless-функции (AWS Lambda, Azure Functions): холодный старт становится «горячим». Важно: для Azure Functions команда рекомендует ReadyToRun, а не NativeAOT.
- Контейнеризированные микросервисы: Маленький размер образа -> быстрая передача по сети и развертывание.
- Системы с ограниченными ресурсами.



Лабораторная: три режима — одна программа

Опубликуйте один и тот же проект тремя способами:

```
dotnet publish -c Release
```

→ JIT (framework-dependent)

```
dotnet publish -c Release -r win-x64 \
```

```
-p:PublishReadyToRun=true → R2R
```

```
dotnet publish -c Release -r win-x64 \
```

```
-p:PublishAot=true → NativeAOT
```

Метрика	JIT	R2R	AOT
Размер (папка)			
Старт, мс (ср.)			
Память (WS)			

Замерьте старт (10 прогонов, Measure-Command), размер папки и working set. Вывод: когда что выбирать.



**Подведём
итоги занятия**

Итоги:

- JIT — стандарт, гибкий и хорошо оптимизированный для долгоживущих процессов.
- ReadyToRun — "апгрейд" JIT для ускорения старта ценой увеличения размера. Практически без недостатков для многих сценариев.
- NativeAOT — специализированное решение для случаев, где критичны мгновенный запуск, малый размер и память, но с серьезными компромиссами.



Результаты вебинара

Оцените насколько к концу занятия вы можете:

1. Понимать разницу между JIT (Just-In-Time) и AOT (Ahead-Of-Time) компиляцией

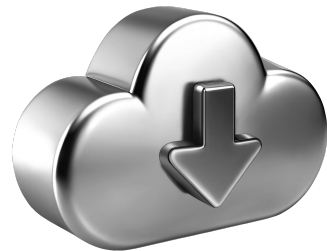
2. Использовать ReadyToRun для ускорения холодного старта приложений

3. Применять NativeAOT для создания нативных самодостаточных исполняемых файлов с минимальным размером и мгновенным запуском.



Список материалов для изучения

1. <https://github.com/dotnet/runtime/blob/main/docs/design/coreclr/botr/readytorun-format.md>
2. <https://learn.microsoft.com/ru-ru/dotnet/core/deploying/native-aot/?tabs=windows%2Cnet10>
3. <https://dotnext.ru/en/archive/2024/talks/a24ece9645844bbea7fc1bf313b359ee/>





Отправьте в чат эмодзи, который отражает ваше настроение после занятия



Продолжите высказывание: теперь я умею/знаю...



Что планируете использовать в работе?

Заполните, пожалуйста, опрос о занятии

Мы читаем все ваши сообщения
и берем их в работу 🖋️❤️

OTUS

